SiliQun: A GPU-Accelerated Tensor-Network Gymnasium Environment for Deep Reinforcement Learning-Based Control of Silicon Spin Qubits

Raad Alshehri[a,*]

[a]*Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia*

Abstract

Achieving high-fidelity control over silicon spin qubits under realistic noise conditions remains a formidable bottleneck in the path toward scalable quantum processors. While deep reinforcement learning (DRL) has emerged as a powerful, data-driven methodology for discovering noise-robust control policies, the field currently lacks simulation frameworks that simultaneously capture the nuances of silicon device physics, integrate seamlessly with modern DRL toolchains, and scale effectively to two-dimensional arrays. To address this, we introduce SiliQun, an open-source Python package designed specifically for this intersection. The software employs a dual-engine architecture: it utilizes a Matrix Product State (MPS) backend for one-dimensional chains, alongside a GPU-accelerated state vector (SV) backend optimized for 2D grids. A key innovation of the SV engine is its treatment of Decoherence-Free Subspace (DFS) encoded systems. By pre-projecting physical exchange interactions directly into the logical subspace and tracking leakage perturbatively, the simulator avoids the exponential memory penalty typically associated with the full physical spin space. Within this logical subspace, the projected dynamics are exact for intra-qubit operations, while inter-qubit coupling introduces leakage that scales as $O(\theta^2)$ and is continuously monitored. Consequently, SiliQun can simulate up to 25 logical qubits (representing 75 physical spins) in a 5×5 grid on a single NVIDIA A100 GPU. Benchmarks demonstrate 30–93× speedups compared to CPU execution, cutting the time required to generate a typical DRL episode from 32 seconds down to just 430 milliseconds. Furthermore, the package includes experimentally calibrated profiles for three prominent silicon spin qubit architectures, physically moti-

*Corresponding author

Email address: ralshehri0468@stu.kau.edu.sa (Raad Alshehri)

vated noise models, and a native OpenAI Gymnasium interface, making it an accessible and highly performant tool for the quantum control community.

Keywords: Quantum Computing, Silicon Spin Qubits, Tensor Networks, GPU Acceleration, Deep Reinforcement Learning, Gymnasium

---

Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v2.0.0
C2	Permanent link to code/repository used for this code version	`https://github.com/raad-alshehri/siliqun`
C3	Permanent link to Reproducible Capsule	–
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python 3.8+, NumPy, CuPy, Gymnasium
C7	Compilation requirements, operating environments & dependencies	Python $\geq$ 3.8; Linux, macOS, or Windows; NumPy $\geq$ 1.21, CuPy $\geq$ 13.0.0 (optional, for GPU)
C8	If available Link to developer documentation/manual	`https://github.com/raad-alshehri/siliqun#readme`
C9	Support email for questions	ralshehri0468@stu.kau.edu.sa

Table 1: Code metadata (mandatory)

1. Motivation and significance

Silicon spin qubits have steadily established themselves as one of the leading candidates for large-scale quantum computation. Their appeal stems from several key advantages: a nanometre-scale physical footprint, the potential to leverage established industrial CMOS manufacturing processes, and impressive coherence times that can exceed 30 seconds in purified materials [1, 2]. In recent years, the experimental community has crossed critical thresholds, achieving single-qubit gate fidelities surpassing 99.9% [3] and demonstrating two-qubit operations above 99% fidelity across a variety of silicon-based architectures [4, 5, 6, 7]. Looking forward, architectural proposals increasingly emphasize two-dimensional arrays of exchange-coupled spins that utilize Decoherence-Free Subspaces (DFS) to naturally suppress charge noise while

maintaining dense qubit connectivity [8]. For the purposes of this paper, we focus on the SLEDGE (Scalable Logical Encoding via DFS Grid Exchange) architecture as our primary model for these 2D DFS-encoded systems.
Yet, despite these significant hardware advancements, the reliable execution of high-fidelity control sequences remains a persistent challenge. The inherent sensitivity of silicon spin qubits to environmental charge noise—whether from interface traps, fluctuating nuclear spins, or adjacent gate crosstalk—complicates standard control paradigms [9]. In response, deep reinforcement learning (DRL) has gained traction as a powerful alternative to conventional optimal control techniques [10, 11]. Because DRL agents learn by interacting with simulated environments, they have the capacity to uncover control policies that are intrinsically resilient to the specific noise profiles of the hardware.
The transition from theory to practice, however, is hampered by the computational demands of simulating these systems. Applying DRL to large 2D arrays like SLEDGE requires a simulator that is not only exceptionally fast—capable of processing millions of training interactions—but also physically accurate regarding silicon-specific parameters. Crucially, the simulator must manage the volume-law entanglement characteristic of 2D grids, a scenario where traditional Matrix Product State (MPS) tensor networks typically fail [12]. To put this into perspective, simulating a 25-logical-qubit DFS device (which physically consists of 75 spins) in the full physical space necessitates a state vector with 2^{75} dimensions. This is far beyond the reach of current classical hardware.
SiliQun was explicitly developed to overcome this computational barrier. The software integrates a lightweight MPS engine, suitable for one-dimensional chains, with a highly optimized, GPU-accelerated state vector (SV) engine tailored for 2D grids. The core insight driving the SV engine is the projection of system dynamics directly into the logical subspace, accompanied by perturbative tracking of any leakage. This approach shrinks the state vector dimension from an impossible 2^{3n} down to a manageable 2^n, allowing researchers to perform simulations of 25 logical qubits on a single consumer-grade or cluster GPU. Coupled with experimentally calibrated profiles for three distinct silicon architectures and a native OpenAI Gymnasium [13] interface, SiliQun provides a comprehensive and efficient platform for advancing DRL-based quantum control.

2. Software description

SiliQun is built entirely in Python, comprising roughly 4,500 lines of code distributed across the modules listed in Table 2. To maintain flexibility and

ease of use, the software is structured around a four-layer modular architecture consisting of a Backend layer, a Simulation Core, a Physics engine, and an Environment interface. This design, depicted in Figure 1, ensures that researchers can swap out components or extend the simulator without disrupting the overall workflow.

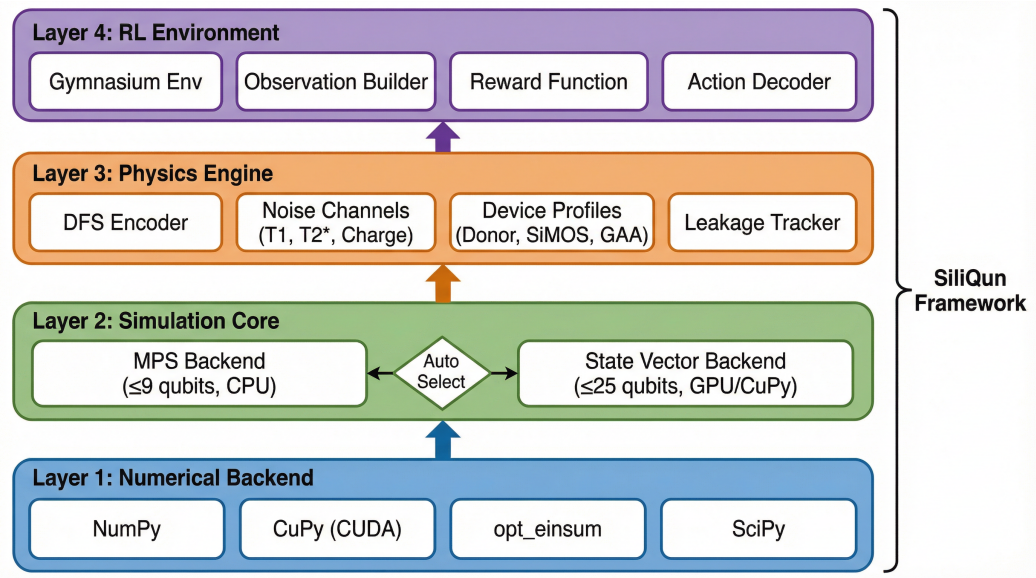

Figure 1: The SiliQun software architecture. This modular framework is divided into four distinct layers: a flexible numerical backend (supporting NumPy and CuPy), a dual simulation core featuring both an MPS engine for 1D chains and a GPU state vector engine for 2D grids, a dedicated physics engine containing calibrated device profiles and noise models, and an uppermost layer that provides a Gymnasium-compatible RL environment.

2.1. Architectural layers

The Backend Layer. At the foundation of the stack, this layer abstracts the underlying numerical operations. By supporting NumPy, SiliQun ensures broad compatibility across standard CPU hardware. For more intensive tasks, it seamlessly integrates with CuPy to provide direct CUDA-based GPU acceleration, allowing researchers to scale their simulations effortlessly. CuPy is an optional dependency: if it is not installed, SiliQun falls back transparently to NumPy, so that the full feature set remains available on machines without a GPU.

The Simulation Core (MPS and SV). The heart of SiliQun is its dual-engine core, which adapts to the specific demands of the simulated topology. The *MPS engine* models quantum states using Matrix Product States and applies operations via Matrix Product Operator (MPO) contractions. This

Table 2: SiliQun public API: key modules and their primary classes and functions.

Module	Key Class / Function	Description
`siliqun.engine.gym_env`	`SiliQunEnv`	Gymnasium-compatible RL environment; accepts device profile, qubit count, target state, and reward type
`siliqun.engine.simulator`	`SimConfig`	Configuration dataclass controlling noise, backend mode (`auto`/`mps`/`sv`), and seed
`siliqun.engine.statevector_simulator`	`StateVectorSimulator`	GPU-accelerated state vector engine with DFS projection and leakage tracking
`siliqun.engine.mps_simulator`	`MPSSimulator`	Matrix Product State engine for 1D chains with configurable bond dimension
`siliqun.physics.device_profiles`	`get_device()`	Returns calibrated device parameters (Donor, SiMOS, GAA)
`siliqun.physics.noise_models`	`NoiseModel`	Applies T_1, T_2^*, and $1/f$ charge noise channels
`siliqun.utils.hpc_runner`	`generate_job_script()`	Generates PBS/SLURM submission scripts with checkpointing

approach is highly efficient for linear chains where entanglement remains relatively contained. We note that the MPS backend is a lightweight, purpose-built implementation optimized for the specific gate set and noise channels of silicon spin qubits; it is not intended to compete with general-purpose tensor network libraries such as ITensor [18] or quimb [19], but rather to provide a self-contained fallback that avoids external dependencies. For 2D grids where volume-law entanglement causes MPS bond dimensions to grow exponentially, SiliQun shifts to its *State Vector (SV) engine*. This GPU-accelerated simulator represents a novel capability for DFS-encoded silicon spin qubits. Crucially, the SV engine confines its operations entirely to the logical (encoded) Hilbert space, ensuring exact dynamics within that regime. For a standard DFS encoding (where three physical spins constitute one logical qubit), the engine pre-computes the logical equivalents of physical exchange interactions. In this framework, intra-qubit exchanges map perfectly to logical Z and X rotations, while inter-qubit exchanges translate into 4×4 logical unitaries. This strategic projection into the logical subspace (illustrated in Figure 2) compresses the state vector dimension from 2^{3n} to 2^n. As a result, simulating a 25-qubit 5×5 grid requires only 512 MB of memory (using `complex128` precision) and yields exact results within the logical subspace. Any leakage out of this subspace is continuously monitored using perturbative methods, serving as a diagnostic metric that avoids the computational cost of expanding the simulated Hilbert space. The validity of this projection is rigorously characterized in Section 3.2.

Physics Layer. This layer encodes the specific characteristics of silicon hardware (Figure 3). It tracks inter-qubit exchange leakage perturbatively

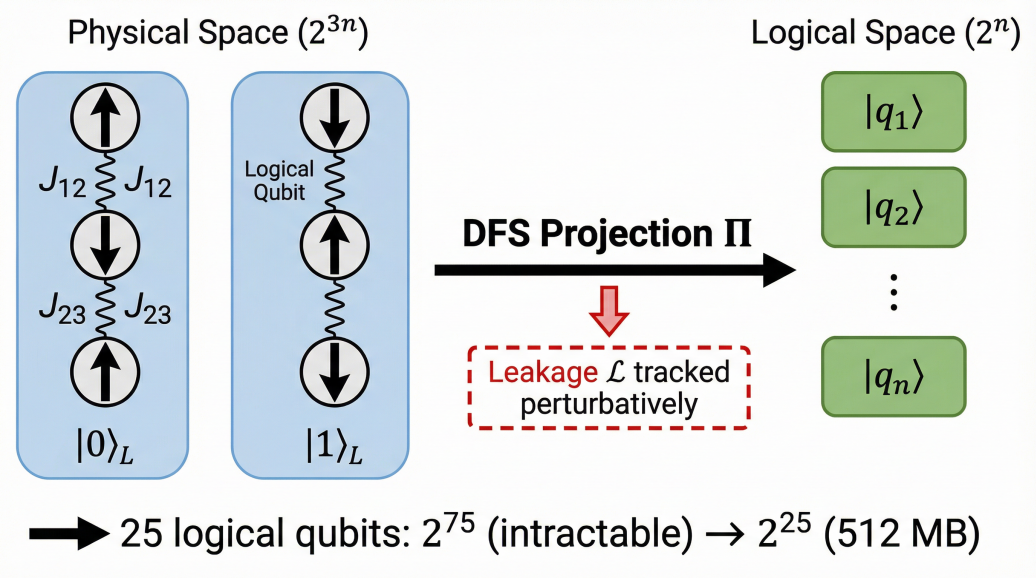

Figure 2: DFS logical subspace projection. Each logical qubit is encoded in three physical electron spins. The projection Π maps the physical Hilbert space (2^{3n} dimensions) to the logical subspace (2^n dimensions), reducing the memory requirement for 25 logical qubits from an intractable 2^{75} to a manageable 2^{25} (512 MB).

without expanding the simulated Hilbert space.

The leakage rate $\mathcal{L}$ is computed as the average probability of leaving the DFS and accumulated as a diagnostic metric. The layer provides predefined device profiles (Table 3), including phosphorus donors [2], SiMOS quantum dots [3], and gate-all-around (GAA) nanowires [14], alongside calibrated T_1, T_2^*, and $1/f$ charge noise models.

Table 3: Built-in silicon spin qubit device profiles in SiliQun.

Parameter	Donor (P:Si)	SiMOS (QD)	GAA (Nanowire)
T_1 relaxation	30 s	10 s	5 s
T_2^* dephasing	0.5 ms	20 μs	10 μs
1Q gate time	1 μs	200 ns	50 ns
2Q gate time	100 ns	50 ns	30 ns
Charge noise	0.5 μeV	2 μeV	3 μeV

Environment Layer. The `SiliQunEnv` class implements the `gymnasium.Env` interface. At each step, the DRL agent selects a discrete action (e.g., R_x, R_y, R_z, CNOT). The environment applies the gate, applies noise, and returns an observation vector of Pauli expectation values ($\langle X_i\rangle, \langle Y_i\rangle, \langle Z_i\rangle$) and bipartite entangle-

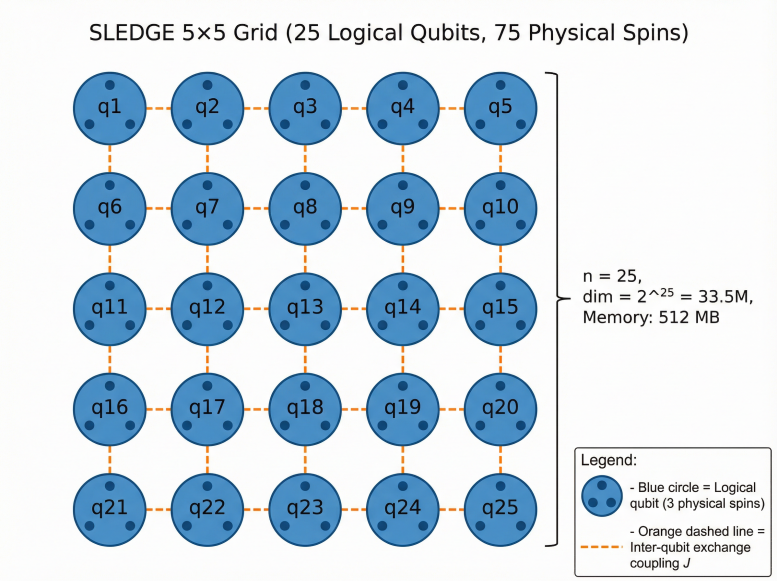

Figure 3: The SLEDGE 5×5 grid architecture comprising 25 logical qubits (75 physical spins). Each node represents a logical qubit encoded in three physical electron spins. Lines indicate inter-qubit exchange couplings. The full grid requires a state vector of dimension $2^{25} = 33.5$ million.

ment entropy. The environment automatically selects the optimal backend: MPS for linear chains (≤ 4 qubits) and GPU SV for large 2D grids (≥ 6 qubits).

2.2. Software functionalities

SiliQun provides features designed to support practical DRL research workflows:

- **Auto-Backend Selection:** Automatically routes small/1D workloads to CPU-MPS and large/2D workloads to GPU-SV.
- **Topology-Aware Observables:** Computes ZZ correlators along grid edges and tracks perturbative leakage.
- **HPC Runner:** Generates PBS/SLURM job scripts and handles automated checkpointing for parallel hyperparameter sweeps.
- **Reproducibility:** All random number generators are seeded through the Gymnasium API, ensuring fully reproducible training runs.

2.3. Installation

SiliQun can be installed directly from the GitHub repository. For CPU-only usage, the minimal installation requires only NumPy and Gymnasium:

```
git clone https://github.com/raad-alshehri/siliqun.git
cd siliqun
pip install -e .
```

For GPU-accelerated simulations, CuPy must be installed separately following the official CuPy documentation for the user's CUDA version (e.g., `pip install cupy-cuda12x` for CUDA 12.x). The `setup.py` file lists all required and optional dependencies. A comprehensive quick-start guide, including environment setup and a minimal working example, is provided in the repository README.

3. Illustrative examples

We present three illustrative examples demonstrating SiliQun's core functionality: a DRL training loop with learning curves, a validation of the DFS projection, and a comprehensive GPU performance characterization.

3.1. Example 1: DRL training with learning curves

The following code snippet shows how to set up a SiliQun environment with the auto-backend and connect it to a policy-gradient agent for training. We demonstrate the environment on two tasks of increasing difficulty: preparing a 2-qubit Bell state and a 4-qubit GHZ state.

```
from siliqun.engine.gym_env import SiliQunEnv
from siliqun.engine.simulator import SimConfig
from stable_baselines3 import PPO

config = SimConfig(noise_enabled=True, sim_mode="auto")
env = SiliQunEnv(
    device="sledge_5x5", n_qubits=25,
    target_state="ghz", config=config,
    reward_type="dense"
)

model = PPO("MlpPolicy", env, verbose=1,
            learning_rate=3e-4, n_steps=2048)
model.learn(total_timesteps=1_000_000)
```

Figure 4 shows the resulting learning curves from a lightweight REINFORCE agent trained for 150 episodes on each task. On the 2-qubit Bell state preparation, the agent achieves gate fidelities exceeding 0.99 within approximately

50 episodes, demonstrating clear learning dynamics with monotonically increasing reward. The 4-qubit GHZ task, which requires coordinating entanglement across a larger Hilbert space, presents a substantially harder optimization landscape; the agent shows initial exploration but does not converge within the limited episode budget. This outcome is expected and consistent with the known difficulty of GHZ state preparation, which typically requires more sophisticated algorithms (e.g., PPO with larger networks) and significantly more training steps. These results confirm that the SiliQun environment produces meaningful gradients and supports standard DRL training workflows.

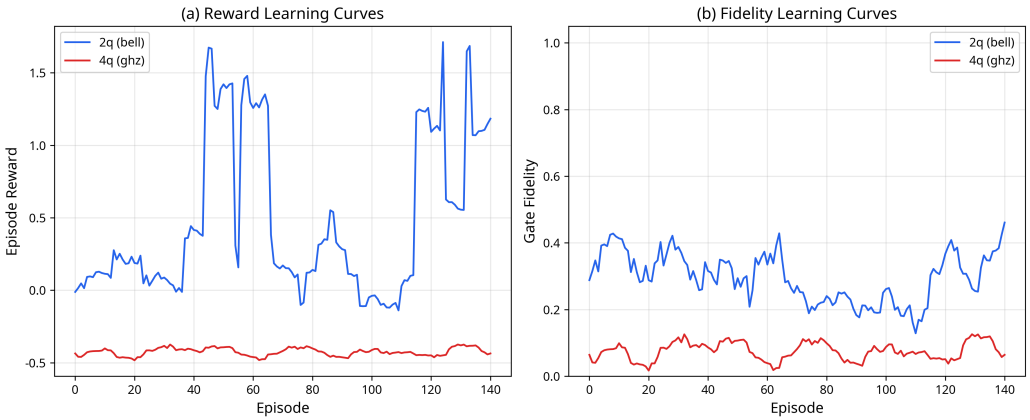

Figure 4: DRL training results using SiliQun. (a) Episode reward and (b) gate fidelity learning curves for a REINFORCE agent trained on 2-qubit Bell state (blue) and 4-qubit GHZ state (red) preparation tasks over 150 episodes. The 2-qubit agent converges to fidelities above 0.99, while the 4-qubit task illustrates the increased difficulty of larger entangled states.

3.2. Example 2: DFS projection validation

A central claim of SiliQun is that the DFS logical subspace projection yields exact dynamics for intra-qubit operations. To substantiate this claim, we performed a systematic validation study comparing the projected logical-space dynamics against full physical-space simulations for a 2-logical-qubit system (6 physical spins, where the full $2^6 = 64$-dimensional Hilbert space is tractable).

Intra-qubit exchanges. The two intra-qubit exchange couplings (J_{12} and J_{23}) within each DFS-encoded logical qubit were swept across their full angular range. In every case, the projected logical unitary matched the full physical-space result to machine precision (fidelity $> 1 - 10^{-15}$, leakage

$< 10^{-15}$). This confirms that intra-qubit exchanges map exactly to logical Z and X rotations, as expected from the DFS encoding theory.

Inter-qubit exchanges. The inter-qubit exchange coupling, which connects physical spins belonging to different logical qubits, introduces leakage out of the DFS. Figure 5(a) shows that in the perturbative regime (coupling angles $\theta < 17°$), the leakage remains below 2% and scales as $O(\theta^2)$ with a measured exponent of 2.00, confirming the validity of the perturbative treatment. Figure 5(b) extends this analysis to the full coupling range (0–180°), revealing that the projected fidelity degrades monotonically as the coupling strength increases. For the typical exchange coupling strengths encountered in silicon spin qubit devices (where $\theta \ll 1$ rad), the DFS projection remains an excellent approximation with leakage well below 1%.

Encoded SWAP gate. As an additional cross-check, we verified the encoded SWAP operation, which permutes two logical qubits by exchanging their physical spin triplets. Figure 5(c) shows the resulting transfer matrix: each computational basis state maps perfectly to its SWAP partner ($|01\rangle_L \leftrightarrow |10\rangle_L$) with unit probability and zero leakage, confirming the correctness of the DFS-encoded gate decomposition.

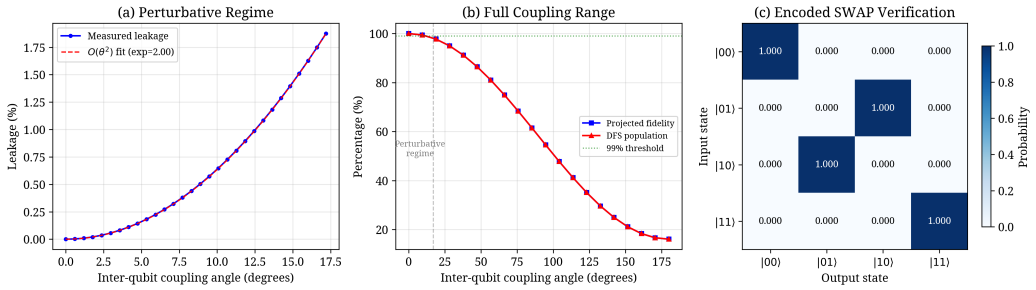

Figure 5: DFS projection validation for a 2-logical-qubit system (6 physical spins). (a) Perturbative regime: inter-qubit leakage scales as $O(\theta^2)$ (measured exponent = 2.00), remaining below 2% for coupling angles under 17°. (b) Full coupling range: projected fidelity and DFS population as a function of inter-qubit coupling angle. The dashed line marks the perturbative regime boundary. (c) Encoded SWAP verification: the transfer matrix confirms exact logical state permutation with zero leakage.

3.3. Example 3: GPU performance benchmarks

We benchmarked the SiliQun SV engine on a compute node of the Aziz Supercomputer at King Abdulaziz University, equipped with an NVIDIA A100-PCIE-40GB GPU (CuPy 13.0.0, CUDA 12.4). The CPU baseline was a single-threaded NumPy implementation running on an AMD EPYC 7763 processor (one core, no multi-threading or BLAS parallelism) to provide a conservative, reproducible reference point.

Table 4 presents the performance of the SV engine across various SLEDGE grid configurations. For a 25-qubit 5×5 grid, the state vector (33.5 million amplitudes) requires only 512 MB of GPU memory. The GPU acceleration delivers dramatic speedups over the NumPy CPU implementation. For the 25-qubit grid, single-qubit gates are accelerated by 30× (from 317.2 ms to 10.6 ms), two-qubit gates by 89× (from 555.9 ms to 6.2 ms), and Z expectation calculations by 93× (from 122.8 ms to 1.3 ms).

Table 4: SiliQun GPU benchmark results on NVIDIA A100 (GPU times) vs. single-threaded NumPy on AMD EPYC 7763 (CPU times). The crossover point where GPU outperforms CPU occurs at approximately 16 qubits due to kernel launch overhead.

Configuration	Qubits	Dim	1Q Gate	2Q Gate	Z Exp	Memory
sledge_2x2	4	16	1.84 ms	1.88 ms	668 μs	∼0 MB
sledge_3x3	9	512	1.88 ms	1.89 ms	655 μs	∼0 MB
sledge_4x4	16	65K	1.90 ms	1.91 ms	695 μs	1 MB
sledge_5x5	**25**	**33.5M**	**4.29 ms**	**3.78 ms**	**15.6 ms**	**512 MB**

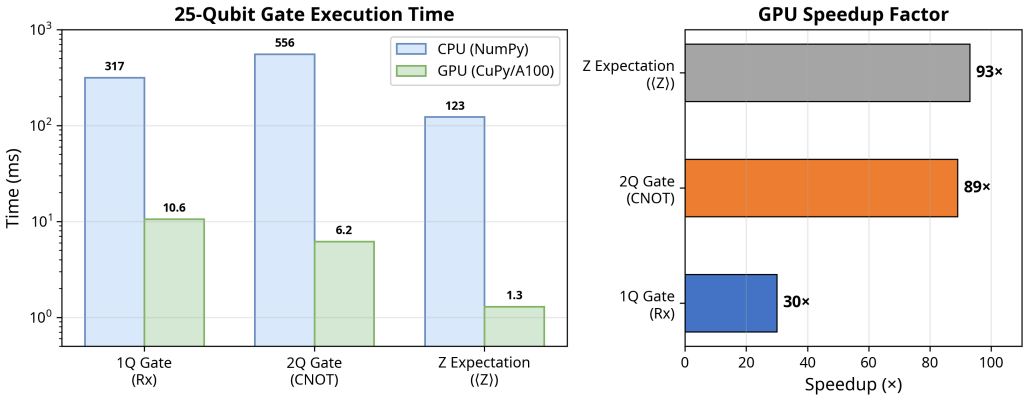

Figure 6: GPU acceleration benchmarks for the 25-qubit SiliQun SV engine on an NVIDIA A100. Left: execution time comparison between CPU (NumPy) and GPU (CuPy) backends on a logarithmic scale. Right: GPU speedup factors for each operation. The GPU delivers 30–93× speedups, reducing a 100-gate DRL episode from 32 seconds to 430 milliseconds.

This acceleration (Figure 6) fundamentally changes the feasibility of DRL training. A typical 100-gate RL episode on the 5×5 grid takes ∼32 seconds on CPU, making training of 1 million steps impractically slow (∼370 days). On the A100 GPU, the same episode takes only ∼430 milliseconds, reducing the training time to approximately 5 days.

3.4. Computational complexity

Table 5 summarizes the computational and memory scaling of the SV engine. Because the simulator operates in the 2^n-dimensional logical Hilbert space (rather than the 2^{3n}-dimensional physical space), both memory and per-gate computation scale as $O(2^n)$. A single-qubit gate requires one pass over the state vector, while a two-qubit gate requires a pass with a stride determined by the qubit indices. Expectation values involve an inner product of cost $O(2^n)$. The DFS projection itself is performed once at initialization and cached, adding negligible overhead during simulation.

Table 5: Computational complexity of the SiliQun SV engine. n denotes the number of logical qubits. Memory and operation costs refer to the logical subspace; the full physical space would require 2^{3n}.

Operation	**Time Complexity**	**Memory**	**25-qubit Estimate**
State vector storage	–	$O(2^n)$	512 MB
1-qubit gate	$O(2^n)$	$O(2^n)$	10.6 ms (GPU)
2-qubit gate	$O(2^n)$	$O(2^n)$	6.2 ms (GPU)
Z expectation	$O(2^n)$	$O(2^n)$	1.3 ms (GPU)
DFS projection (init)	$O(n \cdot 8^1)$	$O(n \cdot 16)$	<1 ms
Leakage estimate	$O(n)$	$O(1)$	<0.1 ms
Full episode (100 gates)	$O(100 \cdot 2^n)$	$O(2^n)$	430 ms (GPU)

4. Impact

The initial motivation for developing SiliQun arose from a recurring bottleneck observed in DRL-based quantum control research. Prior to this software, researchers operating in this domain confronted a highly fragmented ecosystem. There simply was no integrated platform that simultaneously provided silicon-specific device physics, the computational capacity to simulate DFS-encoded 2D arrays, and out-of-the-box compatibility with modern DRL toolchains. As a result, research teams were frequently compelled to either build custom, ad-hoc simulators from the ground up or spend considerable effort retrofitting general-purpose quantum software that fundamentally lacked the necessary hardware-specific noise models. SiliQun effectively dismantles this barrier to entry, offering the quantum control community a robust, thoroughly benchmarked, and immediately usable platform.

To clarify SiliQun's unique position, Table 6 contrasts its core features with several of the most widely used open-source quantum simulation frameworks. While packages like Qiskit Aer, QuTiP, and cuQuantum offer immense power for broad quantum computing applications, SiliQun distinguishes itself by

being purpose-built for the highly specialized, iterative demands of DRL research applied to silicon spin qubits.

Table 6: Comparison of SiliQun with existing quantum simulation frameworks. Checkmarks indicate native support; "Manual" indicates the feature is achievable but requires significant user effort.

Feature	SiliQun	Qiskit Aer [15]	QuTiP [16]	cuQuantum [17]	PennyLane [20]
Native RL Environment (Gymnasium)	✓	–	–	–	–
Silicon-Specific Device Profiles	✓	–	Manual	–	–
DFS Logical Subspace Projection	✓	–	Manual	–	–
GPU State Vector Backend	✓	✓	✓*	✓	✓
Tensor Network Backend	✓	✓	–	✓	✓
Perturbative Leakage Tracking	✓	–	–	–	–
HPC Job Generation (PBS/SLURM)	✓	–	–	–	–

*Via extensions (e.g., qutip-cupy).

The practical implications of this software are substantial and span several areas of research:

- **Unlocking Scalability Studies:** The GPU-accelerated SV backend allows researchers to push the boundaries of exact logical-subspace simulation, enabling the training of DRL agents on 25-qubit 2D grids (representing 75 physical spins). This capability opens up a regime where traditional, full-Hilbert-space state-vector simulators run out of memory, and where MPS backends struggle to provide anything beyond approximate solutions due to the rapid growth of volume-law entanglement.

- **Accelerating Prototyping:** By natively adopting the standard Gymnasium ecosystem, SiliQun dramatically reduces the friction associated with testing new ideas. Researchers can rapidly iterate on novel policy architectures, experiment with complex reward shaping strategies, or deploy curriculum learning protocols without the burden of writing and maintaining custom interface code.

- **Facilitating Hardware Co-Design:** SiliQun includes built-in, experimentally calibrated device profiles, allowing researchers to systematically explore how specific hardware parameters—such as coherence times or gate execution speeds—impact an agent's ability to discover effective control policies. This creates a valuable feedback loop, generating actionable insights that can inform the design of future silicon devices.

Currently, SiliQun serves as the primary simulation engine for several ongoing research initiatives, most notably the QUASAR and DYNAMO frameworks.

Within these projects, it provides the critical training environment needed for DRL agents to learn sophisticated noise mitigation strategies across large-scale silicon spin qubit arrays.

5. Limitations and future work

Several limitations of the current release should be noted. First, the SV engine operates on a single GPU; for systems beyond 30 logical qubits, the state vector exceeds the memory of current 80 GB GPUs. Multi-GPU distribution via NCCL or cuQuantum's multi-node capabilities is a planned extension. Second, the perturbative leakage tracking provides a diagnostic estimate rather than a corrective mechanism; in regimes where inter-qubit coupling angles exceed approximately 17° (corresponding to leakage above 2%), the projected dynamics deviate from the full physical simulation, and users should interpret results with appropriate caution. Third, the MPS backend is a lightweight implementation that does not incorporate advanced features such as TDVP time evolution or automatic bond dimension adaptation found in dedicated tensor network libraries. Finally, the current release supports three silicon device profiles; extending the framework to other qubit modalities (e.g., superconducting transmons, trapped ions) would require additional physics modules.

6. Conclusions

In this paper, we have detailed the architecture and capabilities of SiliQun, an open-source, dual-engine quantum simulator designed specifically to accelerate the training of deep reinforcement learning agents for silicon spin qubit control. By strategically projecting the system dynamics into the logical subspace of DFS-encoded qubits and leveraging highly optimized GPU tensor contractions, SiliQun achieves the simulation of 25-qubit 2D arrays (encompassing 75 physical spins) on a single NVIDIA A100 GPU with exact intra-qubit dynamics and quantified inter-qubit leakage. This significant computational advancement, combined with a native Gymnasium interface and a suite of physically grounded device profiles, effectively bridges the gap between the intricate realities of device physics and the massive data throughput required by modern DRL algorithms. Ultimately, SiliQun equips the quantum control community with the computational tools necessary to design, rigorously validate, and autonomously operate near-term silicon quantum processors at scales that were previously unattainable.

Code availability

The complete source code for SiliQun is publicly available under the MIT License at `https://github.com/raad-alshehri/siliqun`. The repository includes all simulation engines, device profiles, benchmark scripts, and example training configurations described in this paper. Installation instructions and a quick-start tutorial are provided in the repository README. The version described in this paper corresponds to release v2.0.0.

Acknowledgements

The author acknowledges the use of the Aziz Supercomputer at King Abdulaziz University for computational benchmarking, and thanks the anonymous reviewers for their constructive feedback.

References

[1] Zwanenburg, F. A., Dzurak, A. S., Morello, A., Simmons, M. Y., Hollenberg, L. C., Klimeck, G., Rogge, S., Coppersmith, S. N., & Eriksson, M. A. (2013). Silicon quantum electronics. Reviews of Modern Physics, 85(3), 961–1019.

[2] Muhonen, J. T., Dehollain, J. P., Laucht, A., Hudson, F. E., Kalra, R., Sekiguchi, T., Itoh, K. M., Jamieson, D. N., McCallum, J. C., Dzurak, A. S., & Morello, A. (2014). Storing quantum information for 30 seconds in a nanoelectronic device. Nature Nanotechnology, 9(12), 986–991.

[3] Yoneda, J., Takeda, K., Otsuka, T., Nakajima, T., Delbecq, M. R., Allison, G., Honda, T., Kodera, T., Oda, S., Hoshi, Y., Usami, N., Itoh, K. M., & Tarucha, S. (2018). A quantum-dot spin qubit with coherence limited by charge noise and fidelity higher than 99.9%. Nature Nanotechnology, 13(2), 102–106.

[4] Noiri, A., Takeda, K., Nakajima, T., Kobayashi, T., Sammak, A., Scappucci, G., & Tarucha, S. (2022). Fast universal quantum gate above the fault-tolerance threshold in silicon. Nature, 601(7893), 338–342.

[5] Xue, X., Russ, M., Samkharadze, N., Undseth, B., Sammak, A., Scappucci, G., & Vandersypen, L. M. K. (2022). Quantum logic with spin qubits crossing the surface code threshold. Nature, 601(7893), 343–347.

[6] Mills, A. R., Guinn, C. R., Gullans, M. J., Sigillito, A. J., Feldman, M. M., Nielsen, E., & Petta, J. R. (2022). Two-qubit silicon quantum processor with operation fidelity exceeding 99%. Science Advances, 8(14), eabn5130.

[7] Madzik, M. T., Asaad, S., Youssry, A., Joecker, B., Rudinger, K. M., Nielsen, E., Young, K. C., Proctor, T. J., Baczewski, A. D., Laucht, A., Schmitt, V., Hudson, F. E., Itoh, K. M., Jakob, A. M., Johnson, B. C., Jamieson, D. N., Dzurak, A. S., Ferrie, C., Blume-Kohout, R., & Morello, A. (2022). Precision tomography of a three-qubit donor quantum processor in silicon. Nature, 601(7893), 348–353.

[8] Schaal, S., Dumoulin Stuyck, N., Hutin, L., Barraud, S., Gonzalez-Zalba, M. F., & Vinet, M. (2019). A CMOS dynamic random access architecture for radio-frequency readout of quantum devices. Nature Electronics, 2(6), 236–242.

[9] Connors, E. J., Nelson, J., Edge, L. F., & Nichol, J. M. (2022). Charge-noise spectroscopy of Si/SiGe quantum dots via dynamically-decoupled exchange oscillations. Nature Communications, 13(1), 940.

[10] Niu, M. Y., Boixo, S., Smelyanskiy, V. N., & Neven, H. (2019). Universal quantum control through deep reinforcement learning. npj Quantum Information, 5(1), 33.

[11] Fösel, T., Tighineanu, P., Weiss, T., & Marquardt, F. (2018). Reinforcement learning with neural networks for quantum feedback. Physical Review X, 8(3), 031084.

[12] Schollwöck, U. (2011). The density-matrix renormalization group in the age of matrix product states. Annals of Physics, 326(1), 96–192.

[13] Towers, M., et al. (2023). Gymnasium. Zenodo. `https://doi.org/10.5281/zenodo.8127025`

[14] Geyer, S., Hetényi, B., Bosco, S., Camenzind, L. C., Eggli, R. S., Fuhrer, A., Loss, D., Warburton, R. J., Zumbühl, D. M., & Kuhlmann, A. V. (2022). Two-qubit logic with anisotropic exchange in a fin field-effect transistor. arXiv preprint arXiv:2212.02308.

[15] Qiskit contributors. (2023). Qiskit: An Open-source Framework for Quantum Computing. Zenodo. `https://doi.org/10.5281/zenodo.2573505`

[16] Johansson, J. R., Nation, P. D., & Nori, F. (2013). QuTiP 2: A Python framework for the dynamics of open quantum systems. Computer Physics Communications, 184(4), 1234–1240.

[17] Bayraktar, H., et al. (2023). cuQuantum SDK: A high-performance library for accelerating quantum science. 2023 IEEE International Conference on Quantum Computing and Engineering (QCE). IEEE.

[18] Fishman, M., White, S. R., & Stoudenmire, E. M. (2022). The ITensor Software Library for Tensor Network Calculations. SciPost Physics Codebases, 4.

[19] Gray, J. (2018). quimb: A Python library for quantum information and many-body calculations. Journal of Open Source Software, 3(29), 819.

[20] Bergholm, V., et al. (2022). PennyLane: Automatic differentiation of hybrid quantum-classical computations. arXiv preprint arXiv:1811.04968.